

Arrays, Linked Lists, and Binary Search Trees

Arrays

An array is a **fixed size**, **homogeneous** linear data structure. All elements are the same type and stored contiguously in memory.

```
std::array<int, 5> a{};
```

0	0	0	0	0
0	1	2	3	4

Big-O Notation

Before we start comparing operations, we need a vocabulary for cost.

Big-O is the **asymptotic upper bound of a function's growth rate**.

Asymptotic means we care about behavior as n approaches infinity — we drop constants and lower-order terms. $O(2n)$ and $O(n + 100)$ are both $O(n)$.

Big-O describes the **worst case** — it is an upper bound. Two related notations fill in the rest of the picture: - **Ω (Omega)** — lower bound (best case). - **Θ (Theta)** — tight bound (best and worst match).

When people say “ $O(n)$ ” informally, they often mean $\Theta(n)$. This document follows that convention.

Consider bubble sort, which is $\Theta(n^2)$ — for every element, we scan the rest:

```
for (size_t i = 0; i < n; ++i) {
    for (size_t j = 0; j < n - i - 1; ++j) {
        if (a[j] > a[j + 1]) {
            std::swap(a[j], a[j + 1]);
        }
    }
}
```

Now we have the language to talk about array operations.

Array Operations

Size and Capacity

Arrays can be extended to track their contents with two numbers:

- **capacity** — max number of elements the array can hold
- **size** — number of elements currently stored

capacity = 5, size = 3

10	20	30		
0	1	2	3	4
^size				

size tells us where the live data ends. Operations like `insert_back` and `remove_back` rely on this.

find — $O(n)$

Linear scan from index 0 to the end.

```
for (size_t i = 0; i < a.size(); ++i) {  
    if (a[i] == value) return true;  
}  
return false;
```

Worst case: scan every element. Best case: value at index 0.

Q: What is the best case for find? Does Big-O capture that? (Answer: $\Omega(1)$. Big-O is just the upper bound; we need Ω for the best case.)

insert_back — $O(1)$

Write at `data[size]`, increment `size`. No shifting needed.

insert_front — $O(n)$

Must shift every element right to make room.

insert 99 at front:

before: [10, 20, 30, 40, __] size=4

step 1: [10, 20, 30, 40, 40] shift `a[3] → a[4]`
step 2: [10, 20, 30, 30, 40] shift `a[2] → a[3]`
step 3: [10, 20, 20, 30, 40] shift `a[1] → a[2]`
step 4: [10, 10, 20, 30, 40] shift `a[0] → a[1]`
step 5: [99, 10, 20, 30, 40] write 99 at `a[0]`

remove_back — $O(1)$

Decrement `size`.

Q: Do we need to erase the value at `data[size]`? Why or why not? (Answer: no — it's outside `size` so logically gone. For non-trivial types you'd want to destruct it, but the slot is "dead" either way.)

remove_front — $O(n)$

Shift everything left.

Array Summary

Operation	Array
insert_front	$O(n)$
insert_back	$O(1)$
remove_front	$O(n)$
remove_back	$O(1)$
find	$O(n)$

Notice the asymmetry: back is cheap, front is expensive. That's because of the shifting.

Motivation: Arrays Are Fixed Size

Arrays require a capacity set at creation time. Too small and you run out of space. Too large and you waste memory. You must know the size upfront.

What if you don't?

One solution is a dynamic array (`std::vector` in C++). But let's look at a different approach: a data structure that allocates and frees elements one at a time as needed.

Linked Lists

What is a Linked List?

A chain of **nodes**. Each node holds a value and a pointer to the next node. No contiguous memory. No fixed size.

```
head
|
[10] -> [20] -> [30] -> nullptr

struct Node {
    int data;
    Node* next;
};
```

Q: How does a linked list differ from an array in memory? (Answer: nodes are scattered on the heap, connected by pointers. Arrays are one contiguous block.)

insert_front — $O(1)$

Create node, point it at head, update head.

```
before: head -> [10] -> [20] -> nullptr
after:  head -> [99] -> [10] -> [20] -> nullptr
```

```
void insert_front(int value) {
    head = new Node{value, head};
}
```

Compare to the array version, which had to shift every element. Linked list touches 1 node.

Q: Why is `insert_front` $O(1)$ for linked lists but $O(n)$ for arrays?

insert_back — $O(n)$

Walk to the last node, attach new node.

```
void insert_back(int value) {
    Node* node = new Node{value, nullptr};
    if (head == nullptr) { head = node; return; }
    Node* curr = head;
    while (curr->next != nullptr) { curr = curr->next; }
    curr->next = node;
}
```

Array did this in $O(1)$ by writing at `data[size]`.

Q: Why does the linked list lose the advantage here? (Answer: no random access. We can only follow `next` pointers from the head.)

remove_front — $O(1)$

Advance **head**, delete old node.

remove_back — $O(n)$

Walk to second-to-last node, delete last, set its **next** = **nullptr**.

find — $O(n)$

Walk forward until value found or end reached. Same as array.

Linked List with Tail

Add a **tail** pointer to track the last node.

```
head          tail
|             |
[10] -> [20] -> [30] -> [40] -> nullptr
```

insert_back becomes $O(1)$: jump to **tail**, attach, update **tail**.

Q: Does a tail pointer make **remove_back** $O(1)$? Why or why not?

It does not. **tail** lets us *reach* the last node immediately, but after deleting it we need to update **tail** to point to the new last node — and finding that requires walking the whole list. There is no way to get to the second-to-last node in $O(1)$ with only a **next** pointer.

This is what motivates the doubly linked list.

Doubly Linked List

Each node has both **next** and **prev** pointers.

```
nullptr <- [10] <-> [20] <-> [30] -> nullptr
```

With both a **tail** pointer and **prev** pointers, **remove_back** becomes $O(1)$: jump to **tail**, follow **prev** to the second-to-last node, update it, and delete the old tail.

Q: What operations become easier with a **prev** pointer? What gets harder? (Answer: easier — backward traversal, **remove_back**, removing a known node. Harder — every insert/remove now has to maintain two pointers instead of one, more memory per node.)

Q: If a doubly linked list has more information, why is **find** not faster? (Answer: still no random access. **prev** just lets you go backward; you still walk one node at a time.)

Linked List Summary

The Doubly LL column assumes both a **tail** pointer and **prev** pointers.

Operation	Array	Linked List	LL + Tail	Doubly LL
insert_front	$O(n)$	$O(1)$	$O(1)$	$O(1)$
insert_back	$O(1)$	$O(n)$	$O(1)$	$O(1)$
remove_front	$O(n)$	$O(1)$	$O(1)$	$O(1)$
remove_back	$O(1)$	$O(n)$	$O(n)$	$O(1)$
find	$O(n)$	$O(n)$	$O(n)$	$O(n)$

Q: We've improved some operations with linked lists. What did we give up? (Answer: random access — **a[i]** was $O(1)$, now it's $O(n)$. Cache locality — nodes are scattered, so iteration is

slower in practice even at the same Big-O. Memory overhead — every element pays for one or two pointers.)

Motivation: **find** is Still $O(n)$

Every structure so far requires us to look at every element in the worst case to find a value. Can we do better?

To achieve this, we need a structure where each comparison tells us which **half** of the data to ignore.

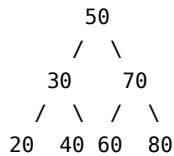
Binary Search Trees

What is a BST?

A tree where each node has at most two children. For every node:

- Left subtree: all values **less than** the node
- Right subtree: all values **greater than** the node

```
struct BSTNode {
    int data;
    BSTNode* left;
    BSTNode* right;
};
```



Built by inserting: 50, 30, 70, 20, 40, 60, 80.

When we compare a search value to 50 and find it's smaller, we discard the entire right subtree in one step. That's the "eliminate half" idea made concrete.

find — $O(\log n)$ average

Compare to current node, recurse left or right.

```
bool find(BSTNode* node, int value) {
    if (node == nullptr) return false;
    if (value == node->data) return true;
    if (value < node->data) return find(node->left, value);
    return find(node->right, value);
}
```

Each step eliminates half the remaining nodes — assuming the tree is balanced.

Why $O(\log_2(n))$?

Look at the *balanced tree* above. Each level holds twice as many nodes as the previous:

```
1 node → height 0 (just root)
3 nodes → height 1
7 nodes → height 2
15 nodes → height 3
```

- A tree of height h holds at most $2^{h+1} - 1$ nodes.
- Flip it around: given n nodes, height h satisfies $2^h \sim n$, so $h \sim \log_2(n)$.
- A search that follows one path from root to leaf takes at most $h = \log_2(n)$ steps.

Each doubling of n adds only one step. That's $O(\log_2(n))$ — dramatically better than $O(n)$.

insert — $O(\log n)$ average

Same traversal as find — walk down until you hit a `nullptr`, then attach.

```
BSTNode* insert(BSTNode* node, int value)
{
    if (node == nullptr)
    {
        return new BSTNode{value, nullptr, nullptr};
    }
    if (value < node->data)
    {
        node->left = insert(node->left, value);
    }
    else if (value > node->data)
    {
        node->right = insert(node->right, value);
    }
    return node;
}
```

The Balance Caveat

The $O(\log n)$ claim depends on the tree being balanced. Insert in sorted order and you get this:

insert 10, 20, 30, 40, 50:

```
10
 \
 20
  \
 30
  \
 40
  \
 50
```

This is a linked list with extra pointers. find is now $O(n)$. **This is the worst case in the BST complexity table.**

Q: How would you avoid this? (Answer: self-balancing trees — AVL, Red-Black. They rotate nodes during insert/remove to keep height $\sim \log n$. Out of scope here, but this is why `std::set` and `std::map` use red-black trees.)

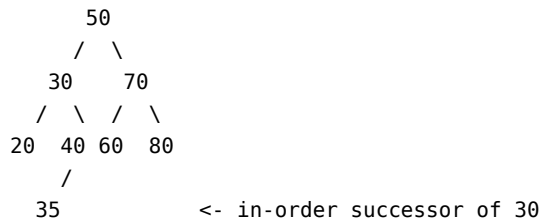
remove — $O(\log n)$ average

Three cases:

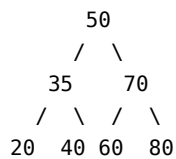
1. **No children** — delete the node.
2. **One child** — replace node with its child.
3. **Two children** — replace with in-order successor, delete successor.

The **in-order successor** of a node is the next value in sorted order — the smallest value still larger than the node. Found by going right once, then left as far as possible. It has at most one child (no left child by definition), so deleting it reduces to case 1 or 2.

Before: remove 30 (has two children)



After:



35's value copied up to replace 30. Original 35 node deleted.
40's left child is now nullptr.

```

BSTNode* remove(BSTNode* node, int value)
{
    if (node == nullptr) return nullptr;

    if (value < node->data)
    {
        node->left = remove(node->left, value);
    }
    else if (value > node->data)
    {
        node->right = remove(node->right, value);
    }
    else
    {
        if (node->left == nullptr)
        {
            BSTNode* tmp = node->right;
            delete node;
            return tmp;
        }
        if (node->right == nullptr)
        {
            BSTNode* tmp = node->left;
            delete node;
            return tmp;
        }

        // two children: replace with in-order successor
        BSTNode* successor = node->right;
        while (successor->left != nullptr) { successor = successor->left; }
        node->data = successor->data;
        node->right = remove(node->right, successor->data);
    }
}

```

```

    return node;
}

```

Note to self: if asked — the recursive call at the end re-searches for the successor, which is slightly redundant. You could unlink it directly when you find it. Same Big-O, cleaner code is a tradeoff.

BST Summary

Operation	Average	Worst
insert	$O(\log n)$	$O(n)$
find	$O(\log n)$	$O(n)$
remove	$O(\log n)$	$O(n)$

Worst case is the degenerate (sorted-input) tree.

Sorted Array

BSTs achieve $O(\log n)$ find by structuring data so each comparison eliminates half. But there's another way to do that: just keep an array sorted.

find — $O(\log n)$

Binary search. Look at the middle. If the target is smaller, recurse on the left half. If larger, recurse on the right half. Each step eliminates half the array.

find 7 in [1, 3, 5, 7, 9, 11, 13]

step 1: middle is 7 → found!

find 11:

step 1: middle is 7, $11 > 7$ → search [9, 11, 13]

step 2: middle is 11 → found!

This is literally the same idea as BST find — compare, eliminate half. A balanced BST is binary search turned into a data structure.

insert — $O(n)$

Binary search finds the position in $O(\log n)$. But making room costs $O(n)$ of shifting. Shift dominates.

Q: If find is $O(\log n)$, why is insert still $O(n)$? (Answer: contiguous memory. To put something in the middle, we have to move everything after it.)

remove — $O(n)$

Same problem in reverse. Find the element fast, then shift to close the gap.

Sorted Array Summary

Operation	Sorted Array	BST
insert	$O(n)$	$O(\log n)$
find	$O(\log n)$	$O(\log n)$

Operation	Sorted Array	BST
remove	$O(n)$	$O(\log n)$

Q: When would you prefer a sorted array over a BST? (Answer: when data is static or rarely changes — you load it once, search many times. Also: better cache locality, lower memory overhead, simpler code. BST wins when there are lots of inserts/removes.)

Overall Comparison

BSTs and sorted arrays don't have a notion of “front” or “back” — they're keyed by value, not position. Hence the dashes in the positional rows.

Operation	Array	Linked List	BST	Sorted Array
insert_front	$O(n)$	$O(1)$	—	—
insert_back	$O(1)$	$O(n)$	—	—
remove_front	$O(n)$	$O(1)$	—	—
remove_back	$O(1)$	$O(n)$	—	—
insert	—	—	$O(\log n)$	$O(n)$
remove	—	—	$O(\log n)$	$O(n)$
find	$O(n)$	$O(n)$	$O(\log n)$	$O(\log n)$

Q: Given this table, when would you reach for each data structure? (Discussion prompt — no single right answer. Push them toward: arrays for known-size sequential data, linked lists for frequent front operations, BSTs for keyed lookup with frequent updates, sorted arrays for keyed lookup of mostly-static data.)

Homework

Implement each of the following data structures in C++.

1. Singly Linked List

Implement a singly linked list of `int` with the following operations:

- `insert_front(int value)`
- `insert_back(int value)`
- `remove_front()`
- `remove_back()`
- `find(int value) -> bool`

Manage memory yourself with `new` and `delete`. Make sure your destructor frees every node; run unit tests with Valgrind and **include the relevant output** in “`linked_list_valgrind.md`”.

2. Binary Search Tree

Implement an unbalanced BST of `int` with:

- `insert(int value)`
- `find(int value) -> bool`
- `remove(int value)`

Handle all three removal cases (no children, one child, two children). Use the in-order successor strategy from lecture for the two-child case. Run unit tests with Valgrind and **include the relevant output** in “`bstt_valgrind.md`”.

3. Sorted Array

Implement a sorted-array container of `int` with a fixed capacity (pick the capacity as a template parameter or constructor argument):

- `insert(int value)` — keep the array sorted
- `find_linear(int value) -> bool`
- `find_binary(int value) -> bool`
- `remove(int value)`

Examine the performance of both `find_` functions and compare them with different array sizes. Report on the results and explain why the results were not necessary what big-O notation would lead you to expect. Run unit tests with Valgrind and **include the relevant output** in “sorted_array_valgrind.txt”.

4. Complexity Analysis (Sorted Array)

For the sorted array specifically, derive and submit the worst-case time complexity of each operation:

- `insert`
- `find`
- `remove`

For each one, State the Big-O complexity and justify it. Lastly, discuss why or why not a sorted array will perform similarly to a BST when values are added in sorted order.

Submission

Submit a single archive containing:

- Source files for all three data structures.
- A test program (or programs) that exercises each operation.
- A short writeup (PDF or plain text) with the complexity analysis for the sorted array.

Note if committee members asked for this and are reviewing the document. For a real assignment, this would be in a GitHub repository (or zip) with a README.md with links to relevant markdown documents (i.e. there would be a markdown file relevant valgrind output for each implementation). There would also be source code already provided for students to fill in the relevant code with unit tests for them to see if their implementations were working. Further, there would be a markdown file for them to fill section 4 for complexity analysis. (That may be LaTeX for math equations, though).